

第二次小作业：卷积网络 报告

王凯灵 521030910356 2023 年 5 月 6 日

问题分析与解决

本次作业使用卷积网络进行 MNIST 数据集进行分类。与此同时，我了解到即使使用 SVM 分类，使用直接把图片展开成向量的方式，在 MNIST 上的精度也可以达到 95% 以上。起初我觉得很诧异，在我对数据集本身进行研究以后我发现这个数据集有十分明显的特点：如图 1 所示，是 MNIST 训练集中 0 和 1 中逐像素统计。要区分 0 和 1，甚至只需要观察某几个像素点的真值（可将 MNIST 图像视为 0-1 真值图）。我在机器学习中进行了尝试，发现只需要 5 个像素点就能达到 100% 的 0-1 分类。对于其他数字也是类似。也就是说，MNIST 也许是一个更适合使用 MLP 直接解决的问题，而不是使用 CNN，RNN。

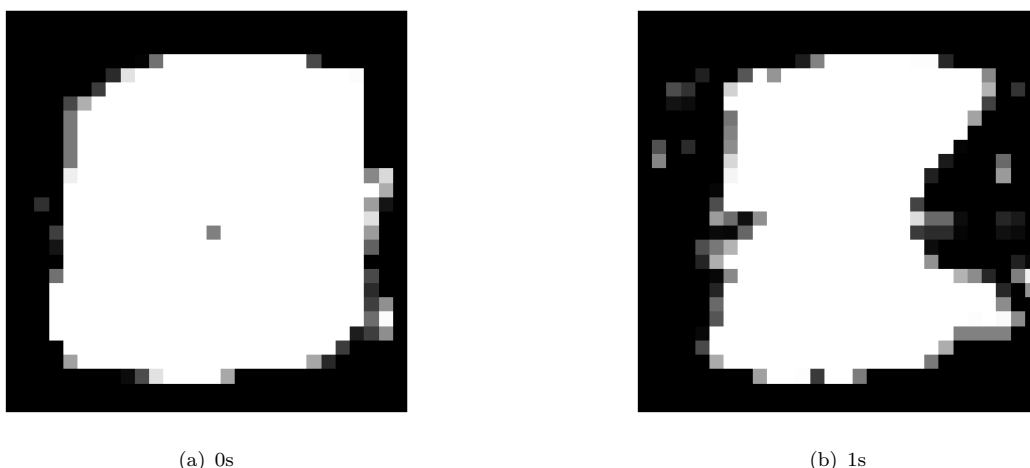


图 1: MNIST 中 0 和 1 的频数统计图

这还意味着按照任务要求分类得到的结果与数据的舍去方法非常有关。MNIST 数据集由来自美国高中生和美国人口普查局的工作人员手写的数字组成，分为前半和后半，其中，一般认为高中生的手写质量更差，如果选取的数据是前面 10% 和后 10%，可能造成显著差异。同时，如图所示，有些人喜欢把 1 往左写，有些人喜欢往右写，而且这种概率不是均等的。如果抽取更多的写法，准确率也会高。换言之，我表述的事实是：再分类精度与随机种子十分有关（如果使用随机抽取）。

实现细节

我使用了两层卷积，卷积核大小为 3，为了防止本来就小的可怜的图片变小，我加了 1 的 padding。卷积核个数分别是 32 和 64。此后，正如我说的，MNIST 数据集可

以只依据很少的特征分类，大多数像素在所有数字中都会写到，或都不会写到，是无用的。我加了一个最大池化。此后是两个线性层，中间层含有 128 个节点。

对于数据集缺失的情况，我按照我的取法，确认数据集大小无误，但精度几乎没有下降。即使如此，我还是尝试做出修改。首先，我不确定任务是否允许增强数据集，因为文件中表述为：试改进你的模型。如果是改进网络模型本身，那么先前的精度也会相应上升。比较合理的方法是给 **loss 按标签赋予权重**，即如果是没筛选的数据集，对应 loss 乘以 0.1。如果允许**增强数据集**，那么对数字进行垂直化以后，重新进行选转即可。这段代码并没有作为参数传入，仅在注释里体现。

分类结果

我事实上没有修改网络结构。在常规分类中，CNN 的结果是 98.23%。在处理完训练集后，结果下降为 97.49%，不到一个百分点。在我使用仅 200 个 epoch，没有学习率下降的简单训练模式中，即使随机种子变化也会导致这种数量的误差（实际上，种子是固定的）。我尝试使用 500epoch 和合适的学习率，结果为 97.53%。可见网络事实上提升空间已经很少。推测即使给缺失的数字加上权重也难以学习有效信息，于是尝试做数据增强。由于这个数据集写的歪七歪八，理想的数据增强应该先通过最大化 X, Y 坐标协方差（即将数字摆正），然后做选转。但直接选转了 -5°, 5° 后，只达到 97.50%。10° 时变为 97.99%。15°, 达到 98.05%，20° 有达到 98.10%，基本接近完整数据集结果。

训练时间在 2 分钟左右，确实调用了 cuda 设备，但不能使用 cuda 设备全部的功耗。如果使用 torch 实现相同的内容，训练时间缩短效果极其明显，使用了完整的功耗，即使当时使用的 torch 并非使用原生的 cuda12。怀疑与 jittor 内部实现有关。此外，发现 jittor 在显存不足时，会长时间卡在调用函数，经过非常久才给出报错信息，且报错内容并没有指示内存不足，而是告知用户失去同步，造成误导。社区有人反映，官方没有回复，在我的建议下，其他用户确认，可能是显存不足导致卡死。总的来说，这是一次十分不愉快的使用体验，计图框架的路还很长。

声明

此作业运行及报告代码 100% 纯手动实现，不含在线参考，任何借鉴和大模型辅助。